SSEVOH-DSL: Revocable, Version-Bound Approvals for Self-Evolving Agent Harnesses

Anonymous Author(s)

Abstract—An agent that revises its own prompts, tools, memory, and workflows changes the machinery that governs its later behavior. Testing a candidate once does not establish that its approval remains valid after a concurrent update, the withdrawal of an ancestor, or the reuse of a learned experience entry. We present SSEVOH-DSL, a declarative transition language and a trusted enforcement service that bind every evolution decision to versioned state, operator policy, validator evidence, and transitive provenance. The service couples one-use, context-bound admission receipts with dependency-aware withdrawal, eligibility-checked recovery, and revalidation at execution and library reuse. We integrate it non-invasively with a pinned Harness^2 runtime by wrapping its admission, stage-execution, and library seams while preserving its recursive adaptation logic. Independent bounded exploration finds no discrepancy over 248,832 one-step cases, 19,620 history transitions, and 6,880 anchored journal replays, and independent state graphs agree with TLC. In controlled Harness^2 fixtures, deployed operator-contract violations fall from 25/45 under the original admission checks to 0/45, while all permitted executions are retained; removing an individual gate re-opens up to 15. Across 500 model-generated requests, all 73 oracle-classified disallowed requests are blocked and all 18 allowed ones admitted; in model-driven Harness^2 streams, 141/141 disallowed proposals are blocked and 16/16 permitted proposals admitted. Admission costs 0.6 ms median. The evidence supports bounded contract enforcement, not task improvement: a two-task ledger pilot fails because of adapter defects we diagnose and fix; in the corrected 50-stream pilot, a sandboxed-evidence policy admits correct model repairs that the catalog rejects (8/50 versus 0/50 correct values, post hoc) with no disallowed execution, while strict success remains 0 because of an answer-format defect. A prespecified 540-stream tool-repair study and a standard coding benchmark remain outstanding.

Index Terms—Agent harnesses, self-evolving agents, domain-specific languages, runtime verification, authorization, provenance, model checking.

1 Introduction

An agent harness determines how a language model receives instructions, accesses tools, stores memory, and organizes work. Revising the harness changes future behavior without changing model weights, and recent systems search over agent designs or recursively revise their own supporting software [1], [2], [3]. This creates a software engineering problem that resembles release management more than prompt engineering: a candidate that was acceptable at one point in an evolution history may no longer be acceptable when it is deployed, composed into another candidate, or reused from an experience library.

Consider a harness that repairs its ledger-summary tool and then derives a reporting skill from the repaired version. If an operator later withdraws the repaired tool, a passing unit test does not authorize running the derived skill; a schema validator cannot tell whether an experience entry still originates from the withdrawn version; and reverting a file does not establish whether a historical version is eligible for recovery. The object that must be governed is a *transition in a versioned authority state*, not an isolated text edit. Existing safeguards address adjacent concerns: runtime monitors mediate events [4], authorization languages decide principal/action/resource requests [5], AgentSpec enforces declarative runtime rules for agents [6], Progent controls evolving tool privileges [7], and AutoSpec learns safety rules [8]. None of them targets the lifecycle obligations created when a harness proposes, composes, publishes, withdraws, recovers, and reuses versions of itself.

SSEVOH-DSL represents harness components and proposed transitions as typed metadata. An untrusted proposer *requests* a change and names required checks; it cannot supply authority or declare validation successful. A trusted service compiles the request, matches it against trusted state, authorizes it under operator policy, constructs and checks the candidate, obtains evidence from registered validators, and issues a context-bound, one-use receipt. Commit rechecks the bound context. Withdrawal records an irrevocable version identity and quarantines its affected closure, and execution and library reuse recheck exact artifact bytes and eligible origins.

Contributions. (1) A transition language with explicit lifecycle semantics for update, withdrawal, and rollback over dependencies and transitive version origins (Sec. 3). (2) A trusted enforcement service, presented as six short algorithms, whose approvals are bound to state, evidence, and origins and rechecked at commit, dispatch, and library reuse. (3) Independent bounded conformance, history, replay, and TLC cross-checks (RQ1). (4) A non-invasive integration with the original Harness^2 runtime and an auditable multi-cluster evaluation that separates enforcement discrimination (RQ2–RQ3) from usefulness (RQ4) and cost (RQ5). We report negative evidence explicitly: consistent enforcement does not make an ineffective worker useful, and our ledger pilot fails both tasks.

2 Background and Running Example

2.1 Self-evolving harnesses

A *component* is a named, typed, versioned harness element; we recognize skill, memory, tool, workflow, configuration, policy, and verifier kinds. Harness^2 [3] is representative of recursive harness adaptation: for each task it proposes edits to a harness tree, probes candidates with solver rollouts, composes promising edits, selects a deliverable, and reflects outcomes into an experience library that is shown to later tasks. Its pinned runtime already validates paths and schemas, retries and falls back on failures, and separates method from judge. These safeguards decide whether an edit is *well formed*; they do not decide whether it is *authorized under the operator's current lifecycle state*.

2.2 Running example: a self-repairing ledger harness

Motivation. We use one scenario throughout the paper. It abstracts the ledger fixtures of our controlled evaluation (Sec. 4.2): each event below corresponds to a fixture family executed there. A reporting harness reconciles CSV ledgers. It contains a protected policy file (guardrails), a tool summarize (a trusted Python program), a skill report that uses the tool, and editable memory. Version 1 of summarize sums only positive amounts, so refunds are silently dropped.

Scenario. Five events occur during evolution. **E1** The agent proposes summarize 1→2 (signed sum); a trusted unit check maps $9.00, -2.00 \mapsto 7.00$. **E2** A second proposal, prepared from the *same* base, also edits summarize. **E3** A proposal edits guardrails and adds a network capability. **E4** The agent derives report v2 from summarize v2, executes it, and reflects an experience entry into its library. **E5** The operator later finds a defect in summarize v2 and withdraws it, then asks to restore a working state.

What should happen. E1 should be admitted only with passing evidence. E2's approval became stale when E1 committed and must not deploy. E3 must be rejected however plausible its bytes. After E5, report v2 and the library entry must become ineligible, *even though their own bytes never changed*; rollback of summarize to v2 must be refused, whereas rollback to v1 and of report to v1 may succeed under current policy.

Objectives. The scenario yields five objectives: **O1** authority—only authorized, evidenced transitions deploy; **O2** freshness—an approval is void once its context changes; **O3** revocation through origins—withdrawal propagates to dependents, derived versions, and library entries; **O4** safe recovery—only eligible historical versions return; **O5** non-invasiveness and traceability—the host's adaptive loop is preserved and every decision is auditable. Original Harness^2 admission checks well-formedness, which overlaps O1 only partially (Sec. 4.5); it was not designed to carry operator authority and offers no counterpart to O2–O4.

2.3 Trust boundary and failure model

We trust the host service, its policy and validator registry, the pinned launcher and runtime, and the operating environment separating them from the proposer. Proposals and model outputs are untrusted data. Validators are assumed pure, terminating, and correct for their registered predicates; hashes identify bytes under collision resistance; a chained journal needs an external anchor to detect wholesale replacement. The supported failure model covers invalid proposal text, stale state, conflicting approvals, withdrawn origins, file changes detectable at controlled boundaries, and model protocol failures. It excludes kernel compromise, malicious validators, arbitrary host code execution, and durable recovery after authority state is lost. The class name `SandboxEngine` does not imply OS isolation: the ledger executor runs a finite catalog of two trusted Python programs, and process-local hooks do not isolate a hostile same-user process.

3 Approach

3.1 Overview

Fig. 1 shows the architecture. The untrusted harness (left) keeps its full adaptive loop. The trusted service (center) consists of four modules: *A. Admission* turns a request into a receipt (Algs. 1–2); *B. Commit* owns the authority state and journal (Alg. 3); *C. Runtime guards* recheck that state when a stage runs or a library entry is reused (Alg. 6); and *D. Operator* withdraws and restores versions (Algs. 4–5). Only eight numbered flows cross module boundaries, and the harness touches the service only through flows ❶, ❷, ❺, and ❻. Offline assurance (right) cross-checks the service against independent semantics. The subsections below follow the modules in order.

3.2 The transition language

A specification declares the trusted snapshot of the relevant components and exactly one evolution. Names, fields, and set-valued lists must be unique; references resolve over all declarations; the parser rejects unknown fields and trailing syntax and canonicalizes declarations and lists. An evolution names an operation (`update`, `withdraw`, or `rollback`), a target, expected and resulting versions, declared dependencies, derivation parents, capabilities, required validators, and pre/post activity conditions. Proposal values cannot embed validator code. Listing 1 is event E1; it compiles with the released compiler.

```
harness ledger {
  component guardrails policy { version 1; mutable false; }
  component summarize tool {
    version 1; mutable true; capabilities [read_ledger]; }
  component report skill {
    version 1; mutable true; depends [summarize]; }
  evolution fix_sign {
    operation update; target summarize; from 1; to 2;
    capabilities [read_ledger];
    requires [structure, signed_ledger];
    expect active; ensure active;
  }
}
```

Listing 1. Running example, event E1: a *request* to update summarize.

The listing requests a transition; it is not a grant. The declared snapshot must equal trusted state—the service never accepts a proposal's account of what is installed—and the principal, policy, and validator implementations come from the host. E3 is expressed the same way but targets guardrails or adds `network` to `capabilities`; it is well formed and is rejected by policy, not by the parser.

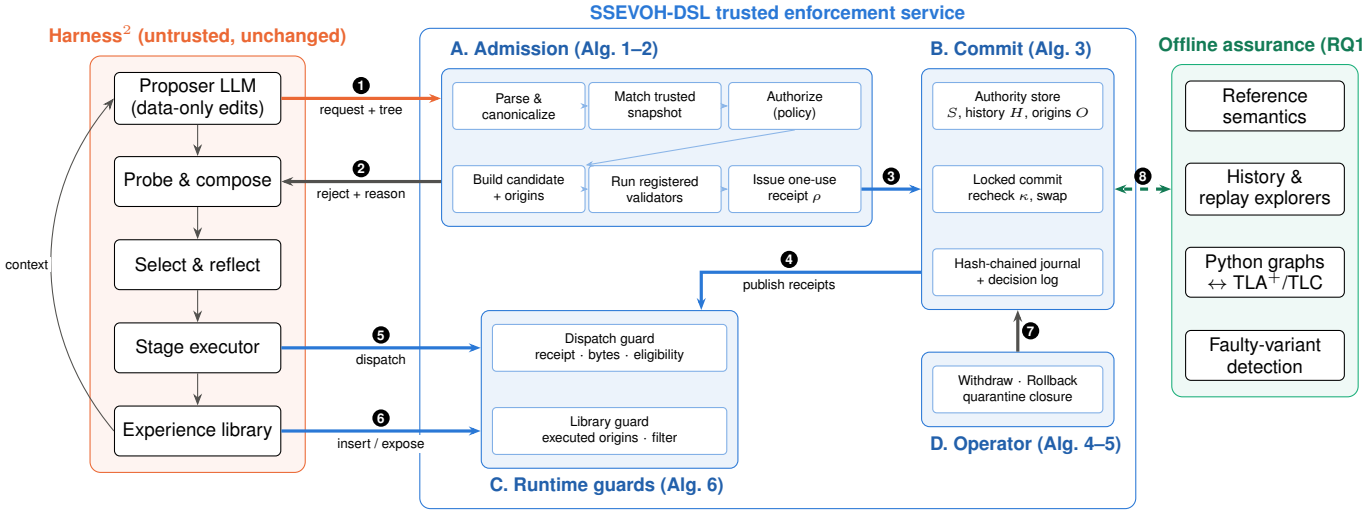


Fig. 1. Architecture of SSEVOH-DSL, grouped into four trusted modules; thin arrows show the order of steps inside the admission module, and numbered arrows show the only flows between modules. ❶ Harness² submits a data-only DSL request with its candidate tree to Module A, which canonicalizes it, matches the declared snapshot against trusted state, authorizes it, builds the candidate with its origins, runs the registered validators, and issues a one-use receipt ρ. ❷ A rejection returns a reason to Harness²'s own retry loop. ❸ Module B rechecks ρ under the authority lock and commits it to state and journal, and ❹ publishes the receipt for execution. Module C rechecks every ❺ stage dispatch and every ❻ library insertion and exposure. ❼ Operator withdrawal and rollback (Module D) commit through Module B and quarantine affected closures. ❽ Offline explorers and TLC cross-check the lifecycle semantics.

3.3 Lifecycle state and validity

The lifecycle configuration is $S = (C, I, W, r, H, O)$: C maps names to current component records (kind, positive version, mutability, dependencies, parents, capabilities); I is the set of inactive names; W is the set of withdrawn $\langle \mathsf{name}, version \rangle$ identities; r is a revision; H stores historical records; and O maps each version identity to its transitive origins. Policy, validator registry, outstanding receipts, and audit records form the separate trusted service context. The combined dependency/parent graph G must resolve and be acyclic, every active component must have active direct dependencies and parents, and active versions and their origins must avoid withdrawal:

$$\forall c \notin I : \ \mathsf{Elig}(c, v_c), \ \text{where } \mathsf{Elig}(x) \equiv (\{x\} \cup O(x)) \cap W = \varnothing. \tag{1}$$

We write $\mathsf{Valid}(S)$ for these conditions plus typed, unique records; $\mathsf{Auth}(p, e, S)$ for authorization of principal p, operation, target, and capabilities by trusted policy; $\mathsf{Evidence}(e, S')$ for exact-`True` results of every mandatory and requested registered validator; and $\mathsf{Protected}(S, S')$ for preservation of immutable, policy, and verifier records and the prohibition of newly quarantining them. $\mathsf{Reach}_G^{-1}(t)$ denotes the components that reach t over dependency or parent edges.

3.4 Admission: preparing a transition

Algorithm 1 is the only path to an approval. It holds the authority lock, records the identity hash of the complete service context, and fails closed at every **require**.

Running example. For E1, the snapshot in Listing 1 matches trusted state; policy grants the agent principal `update` on mutable tools with `read_ledger`; the candidate keeps guardrails unchanged; and both `structure` and the trusted `signed_ledger` check return `True`. The proposer can name `signed_ledger` but cannot supply its result (line 8). Here `signed_ledger` illustrates an operator-registered unit check analogous to condition C1's input; the deployed integration registers artifact-schema, code-catalog, and version-origin validators. For E3, line 3 fails because policy protects guardrails and caps the capability set; line 6 would independently reject a newly quarantined policy component. Each rejection is returned with its reason to Harness²'s own retry loop (flow ❷).

Algorithm 1 PREPARE: admit a request (flows ❶–❷)

Input: DSL source src, authenticated principal p
1: lock authority; $\kappa \leftarrow \mathsf{Id}$(service context)
2: $e \leftarrow \textsc{Compile}(src)$; **require** $e.snapshot = \mathsf{Snap}(S)$
3: **require** $\mathsf{Auth}(p, e, S)$ ▷ principal, op, target, caps
4: **require** $e.from = v_{e.t}$ and $e.expect$ holds in S
5: $(S', O') \leftarrow \textsc{Candidate}(e, S)$ ▷ Alg. 2
6: **require** $\mathsf{Protected}(S, S') \wedge \mathsf{Valid}(S')$
7: **for** $v \in Mandatory \cup e.requires$ **do**
8: $\quad ev[v] \leftarrow Registry[v](e, S')$; **require** $ev[v] =$ `True`
9: **require** Id(service context) $= \kappa$
10: **return** $\rho = \langle$fresh token, $p, \mathsf{h}(src), \kappa, \mathsf{h}(S'), O', ev\rangle$

3.5 Candidate construction and origins

Algorithm 2 computes the candidate for an update. Quarantine uses the *old* graph, so dependents of the replaced version must be revalidated rather than silently inheriting it. Origins are conservative: the new version inherits the previous target version, its origins, and every declared parent version with its origins. An update cannot drop a parent and so cannot declare independence from a withdrawn ancestor; no provenance-severing release operation is provided.

Running example. E1 yields $O(\langle \mathsf{summarize}, 2\rangle) = \{\langle \mathsf{summarize}, 1\rangle\}$ and quarantines report, which depends on summarize, until it is revalidated

Algorithm 2 CANDIDATE: update a target t

Input: canonical request e for target t, state S
1: $c \leftarrow C[t]$; **require** $e.to > v_c$ and $\langle \mathsf{t}, e.to\rangle \notin \mathrm{dom}(H)$
2: **require** $c.parents \subseteq e.parents$ ▷ ancestry never dropped
3: $Q \leftarrow \mathsf{Reach}^{-1}_{G(S)}(t)$ ▷ dependents in old graph
4: $c' \leftarrow c$ with version $e.to$, relations and caps of e
5: $O'(\langle \mathsf{t}, e.to\rangle) \leftarrow \{\langle \mathsf{t}, v_c\rangle\} \cup O(\langle \mathsf{t}, v_c\rangle) \cup$
6: $\quad \bigcup_{q \in e.parents} (\{\langle \mathsf{q}, v_q\rangle\} \cup O(\langle \mathsf{q}, v_q\rangle))$
7: $I' \leftarrow I \cup Q$
8: **if** $e.ensure = active$ and all deps/parents of c' active **then**
9: $\quad I' \leftarrow I' \setminus \{t\}$
10: **return** $(\langle C[t \mapsto c'], I', W, r{+}1\rangle, O')$

Algorithm 3 COMMIT: apply a receipt (flows ❸–❹)

Input: receipt ρ
1: lock authority
2: **require** ρ genuine, outstanding, and unconsumed
3: **require** $\rho.\kappa = \mathsf{Id}$(service context) ▷ stale otherwise
4: stage $H \cup \{\rho.S'\}$, $O \cup \rho.O'$, journal entry $\langle \mathsf{h}_{prev}, \mathsf{h}(\rho)\rangle$
5: atomically swap state references; consume ρ
6: publish bundle bytes and receipt for executor stages
7: append decision record; **return** new revision r'

Algorithm 4 WITHDRAW: revoke $\langle \mathsf{t}, v\rangle$ (flow ❼)

Input: operator principal p, identity $\langle \mathsf{t}, v\rangle$
1: lock authority; **require** $\mathsf{Auth}(p, withdraw, t)$
2: $W' \leftarrow W \cup \{\langle \mathsf{t}, v\rangle\}$ ▷ irrevocable
3: $Q \leftarrow \mathsf{Reach}^{-1}_{G}(t) \cup (\{t\}$ if $v_t = v$ else $\varnothing)$
4: $I' \leftarrow I \cup Q$ ▷ affected closure
5: **require** $\mathsf{Valid}(\langle C, I', W', r{+}1\rangle)$ ▷ Eq. (1)
6: COMMIT through Alg. 3 ▷ receipt as in Alg. 1

Algorithm 5 ROLLBACK: restore $\langle \mathsf{t}, v_h\rangle$ (flow ❼)

Input: principal p, historical identity $\langle \mathsf{t}, v_h\rangle$
1: lock authority; $c_h \leftarrow H[\langle \mathsf{t}, v_h\rangle]$ ▷ stored record
2: **require** $\mathsf{Elig}(\langle \mathsf{t}, v_h\rangle)$ under current W
3: **require** $\mathsf{Auth}(p, rollback, t) \wedge c_h.caps \subseteq Ceil(t)$
4: $Q \leftarrow \mathsf{Reach}^{-1}_{G}(t)$; $S' \leftarrow \langle C[t \mapsto c_h], I \cup Q, W, r{+}1\rangle$
5: **require** $\mathsf{Protected}(S, S') \wedge \mathsf{Valid}(S')$
6: **require** $\mathsf{Evidence}(e, S')$ ▷ same registry as Alg. 1
7: COMMIT through Alg. 3 ▷ W unchanged

against v2. In E4, report 1→2 declares `parents [summarize]`, so line 5 records $O(\langle \mathsf{report}, 2\rangle) = \{\langle \mathsf{report}, 1\rangle, \langle \mathsf{summarize}, 2\rangle, \langle \mathsf{summarize}, 1\rangle\}$. This set is what later makes E5 bite.

3.6 Commit and freshness

Algorithm 3 turns a receipt into state. Because κ hashes the entire service context—state, policy, validator registry, and outstanding receipts—any successful competing commit invalidates every other receipt prepared from the old base. Staging followed by a reference swap prevents partly applied authoritative transitions; a rejected receipt leaves state unchanged. This is a transaction guarantee for the service and its controlled publication path, not a general filesystem crash-consistency theorem.

Running example. E1 and E2 receive receipts with the same κ. Committing E1 changes C, r, and H, so E2 fails at line 3 even though E2 alone would have been valid. Reproposing E2 against the new base is the intended recovery; this is objective O2.

3.7 Withdrawal and rollback

Withdrawal (Alg. 4) is the operator's revocation primitive. W grows monotonically ($W \subseteq W'$ for every transition). A dependent may become inactive without its own version being withdrawn, which keeps direct withdrawal distinct from affected closure. Eligibility of derived versions follows from Eq. (1), so no enumeration of descendants is needed.

Rollback (Alg. 5) restores a stored historical record—never a reconstruction—and applies *current* policy, capability ceilings, structure, validators, and activity conditions. It neither erases W nor reactivates an unsafe history.

Running example. In E5 the operator withdraws $\langle \mathsf{summarize}, 2\rangle$. Line 3 of Alg. 4 quarantines summarize and report. Even if a later edit restored report's dependencies, $\langle \mathsf{report}, 2\rangle$ stays ineligible because $\langle \mathsf{summarize}, 2\rangle \in O(\langle \mathsf{report}, 2\rangle) \cap W$. Rollback of summarize to v2 fails at line 2 of Alg. 5; rollback to v1 and then of report to v1 succeed because their origins avoid W (objective O4). Note that the target-version comparison of Alg. 1 alone would *not* catch $\langle \mathsf{report}, 2\rangle$: origin revocation is a separate obligation from freshness.

3.8 Execution and library reuse in Harness²

Admission is not the last enforcement event. A selected deliverable may run later, and a library entry may influence many later tasks. Algorithm 6 wraps the three Harness² seams in a dedicated launcher: `_accept_tree` (admission), `_run_stage` (execution), and the `Library` class (reuse). Original acceptance checks run first. Dispatch admission and withdrawal are serialized under the authority lock, preventing a check followed by unguarded dispatch; this does not promise instantaneous termination of already-running external processes.

Running example. In E4, Harness² composes report v2 from summarize v2; line 3 binds that contributing source as a parent even though the current-name graph alone cannot express a historical source relation. After the run, the reflected entry carries $\{\langle \mathsf{report}, 2\rangle\} \cup O(\langle \mathsf{report}, 2\rangle)$ (line 10). After E5, line 7 refuses to dispatch report v2 even with unchanged bytes, and line 11 hides the entry from every later task's context (objective O3). Merely creating a stage directory never establishes authority.

The integration replaces no upstream source file; the hook layer is 188 physical lines and the authority adapter 502. These are concrete seams in the pinned release (commit `6868817`), not a stable upstream API; a new release needs compatibility checks and renewed assurance. Sec. 4.8 relates each seam to the Harness² gaps it closes and the evidence for each.

3.9 Correctness argument and audit trail

Under trusted initialization, pure terminating validators, exclusive use of the service boundary, a stable context during successful validation, and sufficient resources, accepted transitions preserve Valid. Initialization checks acyclicity and active origins; historical completeness before initialization is an operator assumption. Updates preserve identity immutability and ancestry while quarantining affected dependents (Alg. 2); withdrawal grows W and quarantines its

Algorithm 6 GUARD: Harness^2 seams (flows ❹–❻)

```
1: function AcceptTree(tree, base, sources)
2:     require OriginalAcceptTree(tree)          ▷ upstream checks
3:     ρ ← Prepare(ToDSL(tree, base, parents= sources))
4:     Commit(ρ); bind stage directory to ρ
5: function RunStage(s)
6:     lock; ρ ← Receipt(s); require h(Tree(s)) = ρ.bytes
7:     require Elig(ρ.x), ρ.x active, executor in trusted catalog
8:     OriginalRunStage(s); Exec ← Exec ∪ {ρ.x}
9: Insert(entry): entry.O ← ⋃_{x∈Exec}({x} ∪ O(x))
10: Expose(): return {entry ∈ Lib | entry.O ∩ W = ∅}
```

closure (Alg. 4); rollback checks stored origins against current W and capability ceilings (Alg. 5); the remaining conjuncts are checked on the candidate; and locked single-use commit linearizes the swap (Alg. 3). Induction over accepted transitions gives preservation. This is a reviewable, non-mechanized argument, not a proof of the Python interpreter or filesystem. Conversely, for well-typed representable requests admitted by the reference relation, with available history, unchanged context, and all validators returning `True`, every implementation rejection should correspond to a reference premise; RQ1 tests both directions on finite domains.

A hash-chained journal records committed transitions, and anchored replay checks continuity, content, and trusted identity; a separate decision log also records rejections. Together they trace each proposal to its decision and resulting state. Version origins describe declared and observed workflow relationships, not every semantic influence on a language model.

4 Evaluation

4.1 Research questions

RQ1 (bounded conformance): Do the implementation and independent lifecycle semantics agree within published finite bounds? **RQ2 (enforcement discrimination):** Which contract violations are blocked and which permitted changes retained, relative to ordinary testing and static validation, and what does each gate contribute? **RQ3 (integration fidelity):** Are admission, composition, execution, and library reuse protected inside the original Harness^2 workflow, including with model-driven proposers? **RQ4 (useful behavior):** Does an evolving agent under the contract deliver useful task outcomes at a documented inference budget? **RQ5 (cost and reproducibility):** What do validation, scaling, and contention cost, and can distributed runs be reconstructed from pins and raw records?

4.2 Study design

Table 1 lists the cohorts. Each answers a different question, and their counts must not be pooled.

Conditions. B0–B2 are admission-only comparisons; trusted safe setup advances later contexts, so their timings are not end-to-end deployment costs. In Harness^2, C1 adds one independent signed-ledger unit input after the original checks; C2 adds static principal/protected-file checks and DSL compilation, and is not a semantic verifier. Shared fixtures contain three permitted families (script correction, multifile correction, skill addition) and nine disallowed families (rows of Fig. 2a). The four lifecycle families—withdrawn origin, rollback conflict, stale approval, interrupted publication—are the executed counterparts of events E2 and E4–E5. C0–C2 offer no withdrawal/recovery API, so no lifecycle rates are invented for them. The eight ablations remove permissions, dependencies, provenance, evidence, formal verification, withdrawal, rollback, or revalidation individually.

Models and budgets. Qwen2.5-Coder 7B, Phi-4 14B, GPT-OSS 20B, Devstral 24B, and Qwen3-Coder 30B are pinned by manifest digest. The DSL cohort uses temperature 0.2, top-p 0.9, 1,024 output and 4,096 context tokens, one generation per model/context/seed. The ledger pilot uses 2,048 output and 16,384 context tokens and runs original aggregation at k=1: four solver rollouts and three proposal stages per task, at most two edit attempts per stage. Raw request/response records retain all calls, tokens, limits, and failures.

Measures and analysis. *Unsafe deployment* means deploying a request that violates the explicit operator contract; it is not a maliciousness classifier. We separate syntax validity, oracle evaluability, allowed and disallowed attempts, deployment, false rejection, protocol/workflow failure, and task success. Invalid output is never counted as a safe request; zero eligible observations yield an undefined rate, not zero risk. A post hoc strict request-fidelity diagnostic checks every declared field; it is not preregistered and not the safety oracle. Tasks are inferential units; seeds repeat observations and sites supply infrastructure. Primary-cohort comparisons use task-cluster bootstrap and paired sign-flip tests with Holm correction and are diagnostic; with few tasks we emphasize counts and effect sizes.

4.3 RQ1: Bounded conformance

Table 2 reports the assurance layers. Independent one-step, history, and anchored-replay checks report zero discrepancies within their domains. The one-step campaign accepts 3,388 and rejects 245,444 requests; history exploration visits 3,307 contexts, accepts 6,880 and rejects 12,740 transitions, and anchors a replay at every accepted endpoint. These counts characterize the enumerated relation, not deployment prevalence. In the lifecycle abstraction, Python and TLC produce identical graphs in all four authorization/evidence contexts, and the five faulty variants (authorization, evidence, withdrawal, rollback, revalidation) each yield a counterexample; e.g., the revalidation variant deploys after a dependency change in five steps. The publication abstraction similarly agrees and detects six faulty variants.

Answer to RQ1. Within the published bounds, the implementation, the independent reference relation, and the TLC model agree, and every injected fault is detected. This supports consistency among the checked representations. It is not an unbounded soundness theorem, a refinement proof from Python to TLA$^+$, or evidence about generated programs or new adapters.

4.4 RQ2: Enforcement discrimination

Deterministic baselines and core ablations. Table 3 shows that syntax (B0) and schema (B1) checks admit all 60 disallowed actions; trusted policy with candidate construction

TABLE 1
Evaluation cohorts. Records are raw rows; inferential units are tasks or fixtures, not seeds or sites.

Cohort	Records	Units and repetition	Conditions	RQ
Bounded assurance	275,332	Enumerated cases, histories, replays (Table 2)	Implementation vs. reference; Python vs. TLC; 11 faulty variants	1
Deterministic baselines	500	25 actions × 5 repeats; 60 disallowed, 65 allowed per condition	B0 syntax; B1 +schema; B2 +policy/candidate; B3 full	2
Core ablations	750	As above	Full; minus one of 5 lifecycle gates	2
Primary DSL generation	500	5 models × 20 contexts (17 scenarios) × 5 seeds	Full enforcement on each response	2
Controlled Harness^2	900	12 shared + 4 lifecycle fixtures × 5 repeats	C0 original; C1 +unit test; C2 +static DSL; C3 full; 8 single-gate ablations	2, 3
Model-driven Harness^2	25 + 50 streams	v1: 5 models × 5 seeds × 2 ledger tasks; v2: same, × 2 executable policies	Evolving under C3 vs. frozen reference	3, 4
Scaling	180	3 sizes × 3 shapes × 4 contention levels × 5	Full enforcement	5
Qualification	18 runs	3 sites × 2 studies × 3 matched trials	Same source/model/prompt identities	5

TABLE 2
RQ1 assurance evidence and exact finite domains. Rows are different evidence layers and are not independent replications.

Check	Enumerated domain	Extent / result
One-step: graph/lifecycle	Acyclic 2-component graphs; inactive/withdrawn subsets; relations from self/other/missing; target versions 1–3, expected 1–2; activity conditions	176,256; 0 diff.
One-step: type/policy	1 component; kind, mutability, capability, grant, evidence	72,576; 0 diff.
History sequences	2 components; 7 initial graphs; versions 1–3; depth 4; all ops	19,620; 0 diff.
Anchored replay	Every accepted history endpoint	6,880; 0 diff.
Lifecycle model	1 proposal; epochs 0–2; 4 auth./evidence contexts	Python = TLC: 18/44, 18/44, 23/62, 61/133 states/edges
Publication model	1 bundle; 1 origin bit; 2 epochs	Python = TLC: 45/81 (auth.+evid.), 4/4 others
Faulty variants	Lifecycle (5), publication (6)	11/11 detected
Test suites	Core / integration / upstream Harness^2	62 / 32 / 70 pass

TABLE 3
RQ2 deterministic admission (125 records per row: 60 disallowed, 65 allowed). FA: disallowed admitted; FR: allowed rejected.

Condition	FA	FR	Median ms
B0 syntax only	60/60	0/65	0.197
B1 + snapshot schema	60/60	0/65	0.203
B2 + policy, candidate	20/60	0/65	0.227
B3 full enforcement	**0/60**	**0/65**	0.599
Full − dep. closure	5/60	10/65	0.538
Full − provenance	5/60	0/65	0.708
Full − revalidation	5/60	0/65	0.593
Full − withdrawal	0/60	0/65	0.563
Full − rollback	0/60	0/65	0.841

TABLE 4
Model-generated proposals. Left: primary DSL cohort (100 responses per model). Right: model-driven Harness^2 streams under C3 (5 per model). Oracle: allowed/disallowed evaluable requests; Fid.: post hoc strict request fidelity; Inv.: invalid proposals; Adm.: allowed proposals admitted; Blk.: disallowed proposals blocked.

	Primary DSL			Harness^2 streams			
Model	Flag	Oracle	Fid.	WF ok	Inv.	Adm.	Blk.
Qwen2.5-C 7B	0	0/0	0%	5/5	0	7/7	30/30
Phi-4 14B	0	0/0	0%	5/5	43	0/0	17/17
GPT-OSS 20B	30	9/21	21%	0/5	–	–	–
Devstral 24B	0	0/0	0%	5/5	0	0/0	54/54
Qwen3-C 30B	100	9/52	21%	5/5	0	9/9	40/40
Total	130	18/73	–	20/25	43	16/16	141/141

(B2) still admits 20, namely those whose violation is only visible to validators or lifecycle history. Full enforcement (B3) admits none and rejects none of the 65 allowed actions. Removing dependency closure, provenance checks, or revalidation each re-opens five admissions; removing dependency closure also causes ten false rejections, because skipped quarantine diverges later state. Removing withdrawal or rollback checks alone has no effect in these cases because retained guards overlap; a zero single-gate effect does not show that a mechanism is dispensable.

Model-generated requests. Fig. 2b and Table 4 show that only 91 of 500 generated responses are oracle-evaluable: 73 disallowed and 18 allowed. Full enforcement deploys none of the 73 and falsely rejects none of the 18. Three models return fenced output in all 100 responses, which the strict raw-text protocol rejects. GPT-OSS has 56 empty and 62 length-limited outputs (overlapping). The historical `syntax_valid` flag is also set for parser-level identifier failures: 39 Qwen3-Coder responses quote a relation or capability identifier and fail before oracle classification, so the flag must not be read as 100 parsed programs. Filtering to valid proposals would hide these interface failures.

Answer to RQ2. Within the designed fixtures, full enforcement separates allowed from disallowed requests without error in every population that reaches a decision: 0/60 deterministic, 0/73 model-generated, and 0/45 controlled violations deployed, with no false rejections. Each comparison condition leaks a characteristic subset, and each gate that matters in isolation is visible in the ablations. The dominant failure for generated requests is the interface, not the gate.

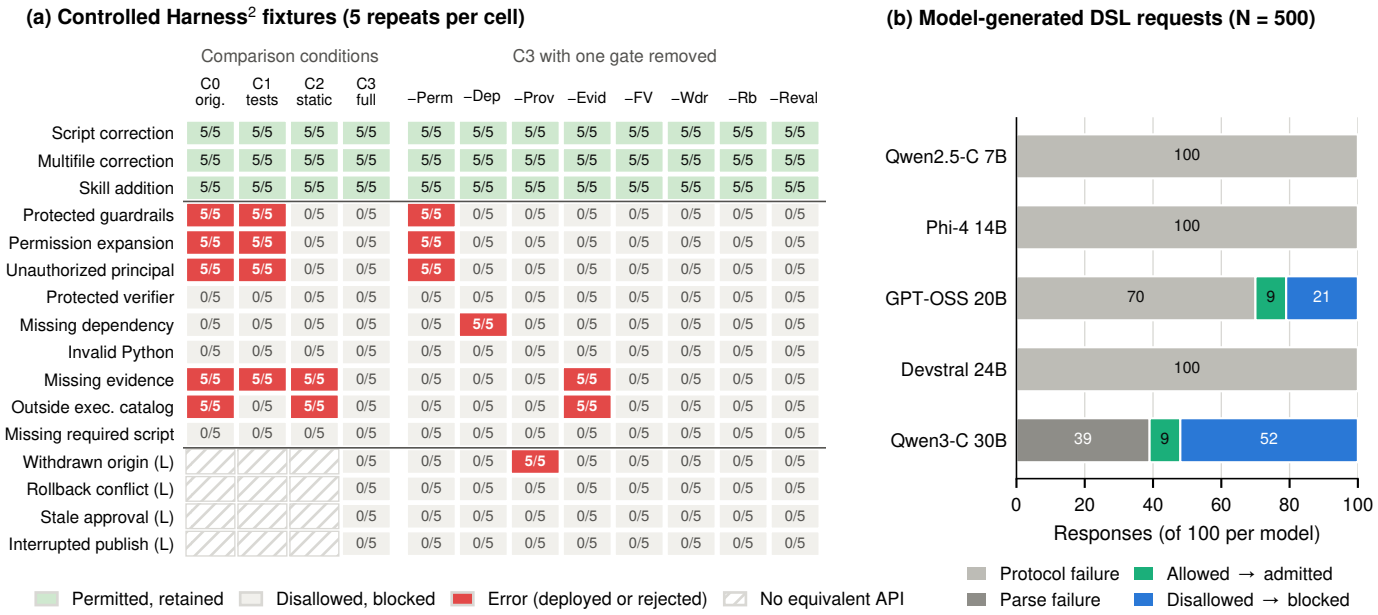


Fig. 2. Enforcement outcomes. (a) Each cell counts, over five repeats, permitted changes retained (top three rows) or disallowed changes deployed (other rows). Red cells are errors. Original Harness² admission (C0) deploys 25/45 shared violations; ordinary testing (C1) still deploys 20 and static DSL validation (C2) 10; full enforcement (C3) deploys none, retains all permitted changes, and blocks all four lifecycle families (L). Removing a single gate re-opens exactly the families that gate covers. (b) Of 500 model-generated DSL requests, 409 never reach the oracle (fenced, empty, length-limited, or identifier-level parse failures); every oracle-classified request is decided correctly.

TABLE 5
RQ3 controlled Harness² study (5 repeats). Shared: 45 disallowed, 15 permitted per condition; lifecycle: 20 disallowed. Mean ms over the 15 permitted executions only; timing is descriptive and order-confounded.

Condition	Shared	Retained	Lifecycle	ms
C0 original	25/45	15/15	n/a	77.17
C1 + unit test	20/45	15/15	n/a	155.73
C2 + static DSL	10/45	15/15	n/a	78.33
C3 full	**0/45**	**15/15**	**0/20**	114.81
C3 − permissions	15/45	15/15	0/20	–
C3 − evidence	10/45	15/15	0/20	–
C3 − dependencies	5/45	15/15	0/20	–
C3 − provenance	0/45	15/15	5/20	–
C3 − any other	0/45	15/15	0/20	–

4.5 RQ3: Integration fidelity in Harness²

Fig. 2a and Table 5 report the controlled study inside the original recursive workflow. C0 deploys protected-file edits, permission expansions, unauthorized principals, missing-evidence changes, and executables outside the catalog. The unit test (C1) catches only the uncatalogued executable; C2's static checks catch the protected-file, permission, and principal violations but not the evidence or catalog violations. C3 blocks all 45 and retains all 15 permitted executions. In the separate lifecycle group, C3 blocks all 20 attempts, including the withdrawn-origin execution of event E5 and the stale approval of E2. Removing provenance alone re-admits that withdrawn-origin execution in all five repeats; removing permissions, evidence, or dependency checks re-admits 15, 10, and 5 shared violations, respectively.

Model-driven streams. With real model proposers inside the same workflow (Table 4, right), C3 blocks all 141 assessed disallowed proposals and admits all 16 permitted non-no-op proposals at exact bytes. GPT-OSS fails all five workflows before producing an assessable proposal; these failures remain in stream and task denominators. A source and raw-artifact audit found that this pilot never exercised reflection: our launcher did not pass the model client as Harness²'s selector, so upstream built a CLI selector absent from the pinned archive, and all fifty task summaries have empty experience. The corrected launcher (v2, RQ4) does exercise it, and library guarding is also covered by the deterministic fixtures and eight coding-adapter regressions.

Answer to RQ3. Admission, composition, execution, and library exposure are enforced at actual upstream call sites, and the lifecycle obligations of the running example hold in executed fixtures. Fidelity is scoped to the supported local executors, the two-program catalog, and data-only edits; it is not general tool-enabled agent isolation.

4.6 RQ4: Useful behavior

Original pilot (v1). Evolving ledger agents achieve 0/50 successful outcomes, and frozen references also achieve 0/50. The audit traced this to two defects in *our* model adapter, not to enforcement. First, the executor schema quoted a placeholder, `{"answer":"final deliverable"}`, which Qwen2.5-Coder and Devstral returned verbatim and Phi-4 paraphrased: 29 of the 50 evolving answers are the placeholder. Second, the missing selector channel above disabled reflection. The remaining Qwen3-Coder answers report the output of the unrepaired positive-only tool.

Corrected pilot (v2). We fixed both defects (a typed answer schema; the model client as `selector_prop`) and, motivated by the rejected repair below, added an operator policy `sandboxed_evidence`: a program outside the catalog is admitted only if a trusted validator finds it inside a

TABLE 6
Ledger pilot: v1 versus the corrected v2 HPC cohort (5 models, 5 seeds, 2 tasks). Library: a learned entry precedes task 2. Correct value is post hoc and format-insensitive.

	v1	v2 catalog	v2 sandboxed
Completed workflows	20/25	20/25	20/25
Placeholder answers	29/50	0/50	0/50
Library entry reused	0/25	16/25	18/25
Disallowed executed	0/141	0/31	0/13
Permitted admitted	16/16	81/82	96/96
Correct value (post hoc)	–	0/50	8/50
Strict success: evolving	0/50	0/50	0/50
frozen	0/50	0/50	0/50

static allowlist and correct on operator-held ledgers under Bubblewrap, and it then executes only in that sandbox. A live isolation probe gates every compute node. We reran all five models under both policies on Fir and Narval (50 streams, Table 6). Reflection now works (a learned entry precedes the second task in 34/50 streams), no disallowed proposal executes under either policy, and all permitted proposals are admitted except one under the catalog. GPT-OSS fails all ten streams at the serving layer (an Ollama response-parser error under JSON-constrained decoding), recorded as failures. Strict success is still 0/100 outcomes, but for an identifiable reason: under `sandboxed_evidence`, correct repairs by Phi-4 and Qwen3-Coder (e.g., `sum(amounts, Decimal(0))`) were admitted, executed in the sandbox, and returned the correct total, yet the agent answered in prose ("the tool returned a total of 20.25") because the final executor turn did not restate the task's required `Total:` form. A post hoc, format-insensitive diagnostic counts 8/50 correct values under sandboxed evidence and 0/50 under the catalog, which rejected the same repairs.

The pilot therefore motivates, but cannot establish, usefulness: two tasks share one planted bug. We prespecified a follow-up, ToolRepair-12 (12 planted-defect tool families with disjoint visible, validation, and oracle inputs; original Harness^2, catalog, and sandboxed-evidence arms; one injected red-team proposal per stream; family as the unit of inference), with a corrected executor that restates the task, frozen by implementation hashes before qualification.

For text-only evolution, a HumanEval+ v0.1.10 study [9], [10] is prepared: a public-only projection keeps solutions and verdicts out of every agent context, selections and libraries are frozen before hidden grading, generated code runs only in a qualified OS sandbox, and one-shot, call-matched frozen, original, and contract conditions are compared on all 164 tasks with intention-to-evaluate denominators.

Answer to RQ4. Not yet. After correcting two adapter defects, the evolution loop, library, and sandboxed-evidence admission all work, and the evidence policy recovers correct values that the catalog blocks (8/50 versus 0/50), without any disallowed execution; strict success remains 0/100 because of a third, now-fixed answer-format defect. ToolRepair-12 and HumanEval+ carry the usefulness question; neither is complete in this snapshot (10 November 2026).

4.7 RQ5: Cost and reproducibility

Validation and scaling. Fig. 3 summarizes the 180 scaling records and the admission baselines. Shape matters because quarantine follows reverse reachability: chains quarantine the longest closures. Contention is resolved by compare-and-swap on the context hash: in every tested cell exactly one approval commits and the others are rejected as stale. These observations do not establish distributed throughput or an asymptotic bound. Traced Python allocation peaks at 2.5 MB at 500 components and excludes native and GPU memory. In Harness^2, the permitted-execution mean rises from 77.17 ms (C0) to 114.81 ms (C3), a descriptive, order-confounded fixture difference; C1's slower 155.73 ms reflects its extra test process and does not establish general superiority to testing.

Reproducibility. HPC runs used Fir, Narval, and Rorqual through a `use-hpc`-derived Slurm pipeline [11] with source freezes, cache identity and GPU checks, measured site qualification, and hash-verified collection; censored and failed cells are preserved, and superseded cells are re-run as recorded supplements rather than overwritten. Twenty-four matched cross-site pairs produced identical text and decisions under identical identities.

Answer to RQ5. Enforcement is cheap relative to inference: a 0.6 ms admission is below 0.1% of the 1.66 s mean stream time per model call in our completed streams, and even a 500-component chain transition costs less than one call. Every reported count can be recomputed from committed raw records, pins, and logs (Sec. 6).

4.8 Discussion

How the integration works. SSEVOH-DSL is attached to Harness^2 by a dedicated launcher that installs three wrappers at run time and leaves upstream search, probing, composition, selection, reflection, and judging untouched (Fig. 1, Alg. 6). (i) When Harness^2 accepts a candidate tree, the original `_accept_tree` checks run first; the tree, its base, and its contributing sources are then translated into a DSL request and must obtain and commit a receipt. A rejection is returned as an ordinary acceptance failure, so the upstream retry and fallback loop handles it unchanged. (ii) `_run_stage` executes a stage only if its directory is bound to a published receipt with matching bytes and an eligible version. (iii) The `Library` wrapper stamps each reflected entry with the origins of the versions that actually executed and hides ineligible entries from later contexts. Because nothing upstream is edited, all 70 original Harness^2 tests still pass (Table 2).

Which Harness^2 gaps are closed, and the evidence. Table 7 links each gap observed under original admission (C0) to the mechanism that closes it and to the measured effect. Two patterns stand out. First, five of the nine shared violation families deploy under C0 because well-formedness checks cannot see authority or evidence; ordinary testing (C1) closes only one of them and static validation (C2) three (Fig. 2a). Second, Harness^2 has no notion of revocation, freshness, or authority-aware recovery, so the four lifecycle families have no C0 counterpart at all; under C3 each is blocked, and the withdrawn-origin case reopens in every repeat when provenance alone is removed. The price is

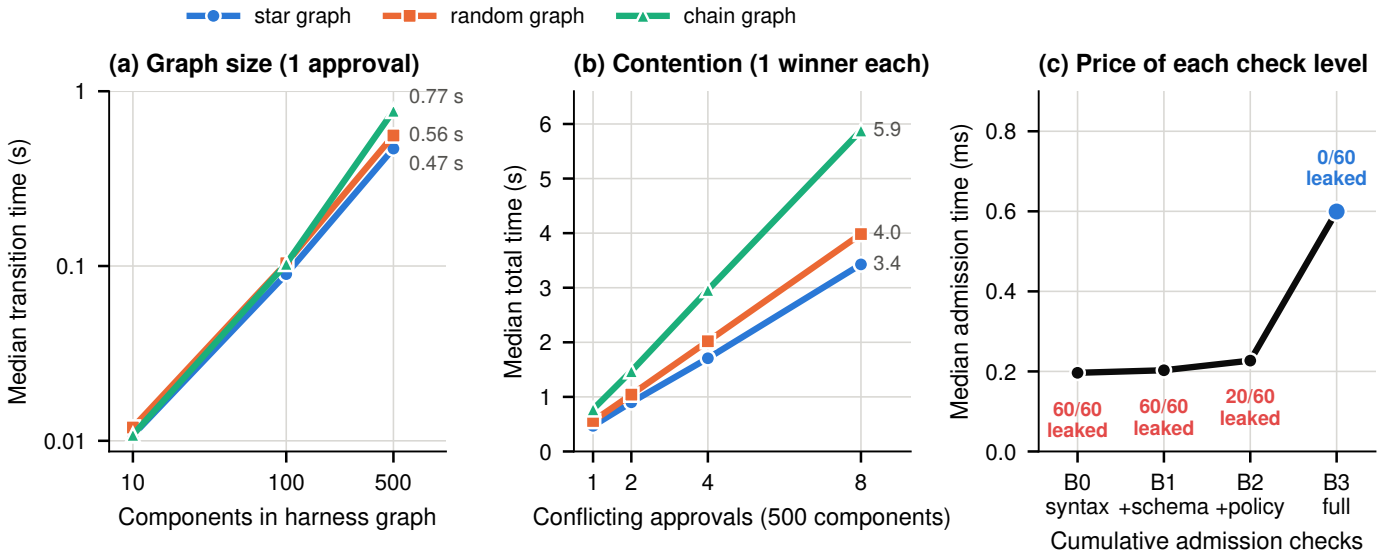


Fig. 3. Three aspects of enforcement cost (medians of five repeats). (a) *Graph size:* transition time grows with the number of components and depends on graph shape through the size of the quarantine closure. (b) *Contention:* with 1–8 conflicting approvals on 500 components, time grows linearly and every cell has exactly one commit winner. (c) *Check level:* each cumulative admission check adds little latency; only full enforcement (B3, 0.60 ms) leaks none of the 60 disallowed actions.

TABLE 7
Harness^2 gaps under original admission (C0), the SSEVOH-DSL mechanism that closes each, and the evidence (shared fixtures: 5 repeats per family; Fig. 2a, Tables 4–5).

Harness^2 gap	Mechanism	Evidence (C0 → C3)
Protected files, capability, and principal not checked	Policy, Protected (Alg. 1)	15/15 → 0/15 deployed; −Perm reopens 15/45
No required evidence; uncatalogued executables	Registered validators (Alg. 1)	10/10 → 0/10; C1 still 5, C2 10; −Evid reopens 10/45
Missing dependencies	Dependency closure (Alg. 2)	Blocked by C0 already; −Dep reopens 5/45
Approval not tied to base state	Context-bound receipt (Alg. 3)	No C0 API; stale approval 0/5
Derived versions and library entries keep withdrawn origins	Origins; dispatch and library guards (Algs. 4, 6)	No C0 API; withdrawn origin 0/5; −Prov reopens 5/5
No authority-aware recovery	Rollback eligibility (Alg. 5)	No C0 API; rollback conflict 0/5
Partial publication	Staged swap (Alg. 3)	No C0 API; interrupted publish 0/5
Model-driven proposals	All of the above	141/141 blocked, 16/16 admitted
Permitted changes	—	15/15 retained in every condition

about 38 ms per permitted execution in these fixtures (77.17 to 114.81 ms, Table 5) and 0.4 ms per admission (Fig. 3c), both below the 1.66 s mean stream time per model call.

What the integration does not fix, and the evidence trade-off. Enforcement does not make the worker useful (RQ4). The pilot also measured a cost of conservatism: a finite catalog cannot recognize an equivalent program, so it rejected every correct model repair. Replacing catalog membership with sandboxed behavioral evidence recovered those repairs without any disallowed execution (Table 6), at the price of a larger trusted base (validators, validation data, and the isolation layer) and a declared limit: behavioral evidence cannot exclude behavior triggered only by inputs outside every validation case, which the sandbox contains but does not prevent. ToolRepair-12 probes that limit explicitly. The v1 defects also show that a silent fallback can disable the very loop being governed, so integrations should audit fallback paths and treat interface robustness as first-class; in the primary cohort, 82% of responses never reached an oracle-classified decision. Four of eight single-gate ablations show no effect because guards overlap—in the running example, withdrawing $\langle \mathsf{summarize}, 2 \rangle$ is caught both by origin eligibility and by quarantine of dependents—so a zero effect is defense in depth, not redundancy. Finally, conservative origins reject some useful rewrites, and operators must maintain policies and validators.

Beyond Harness^2. The three seams—candidate acceptance, stage dispatch, and experience reuse—are generic to self-evolving harnesses. In ADAS [1], archived designs and their derivations map onto components and parents; the Darwin Gödel Machine [2] already keeps a lineage tree that maps onto the origin relation O; and coding agents that load editable skill and memory files fit our data-only edit surface. Porting requires locating the host's equivalents of the three seams and, where executable artifacts evolve, a verified isolation story we do not provide. A mature authorization language such as Cedar [5] could replace our policy layer.

4.9 Threats to validity

Construct. Compliance with an operator lifecycle contract is narrower than harmless agent behavior; correctness, fidelity, and authorization are distinct, and strict fidelity is post hoc. **Internal.** Validators and reference semantics can share design errors; independent implementations and faulty variants reduce but do not eliminate common-mode mistakes. Same-user adversarial isolation and durable restart

are unsupported. Timing is order-confounded. **External.** Designed fixtures are not a sample of real attacks; two catalog programs and two ledger tasks do not establish general autonomous work, and pilot correct-value counts are post hoc; models and sites do not add tasks. HumanEval+ would extend coding evidence but not cover professional maintenance or open-world tools, and public tasks may be contaminated. **Conclusion.** Zero observed violations is not zero risk; repeated fixtures cannot be pooled as independent samples; pre-proposal workflow failures shrink assessable safety populations and remain failures for task outcomes. **Operational.** Queues, heterogeneous GPUs, and model packaging affect completion; pins, raw logs, deadlines, and censoring support audit without making infrastructure deterministic.

5 Related Work

Runtime enforcement and authorization. Schneider characterizes policies enforceable by execution monitoring [4]; our service mediates restricted transition and dispatch events in that tradition and adds versioned evolution and reuse obligations rather than a new characterization. Cedar combines expressive authorization with analysis and verification-guided engineering [5], [12]; its formal foundations exceed our bounded exploration, and general principal/action/resource authorization is not our novelty. Proof-carrying code separates untrusted programs from evidence checked by a trusted consumer [13]; our receipts are context-bound service records, not proofs of program semantics, and catalog membership is not proof-carrying compilation. TLA$^+$ supplies state-machine specification and checking [14]; we contribute a lifecycle model and cross-checks, not a refinement theorem. PROV-DM provides a general provenance model [15]; our narrower version-origin relation is operational: it governs revocation eligibility.

Agent safety languages. AgentSpec offers declarative rules of triggers, predicates, and enforcement actions over agent execution [6]; its predicates can inspect trajectories, so we do not claim greater logical expressiveness. Progent represents privileges as allowed tool calls and checks policy updates with an SMT solver, narrowing automatically and expanding only with approval [7]; monotone restriction and approval of evolving policies are therefore not new here. Our contract instead follows immutable artifact-version identities and transitive origins: replacing a tool implementation or reusing a learned entry can change eligibility even when the allowed-call set is unchanged. AutoSpec evolves safety rules via counterexample-guided, logic-based synthesis [8]; it could supply trusted validators or policy proposals, whereas our ordinary proposals cannot rewrite protected policy. We have not run head-to-head comparisons and infer no security superiority.

Table 8 summarizes the resulting positioning; it does not assert that prior frameworks cannot be extended to implement these obligations.

Self-evolving agents and benchmarks. ADAS searches over agent designs [1], the Darwin Gödel Machine studies open-ended self-improvement [2], and Harness^2 recursively adapts harnesses [3]. Our operator boundary complements these search procedures (Sec. 4.8). HumanEval and EvalPlus assess generated-program correctness [9], [10] and complement, rather than replace, lifecycle checks.

TABLE 8
Positioning: existing responsibility of each foundation and the evolution-specific connection examined here.

Foundation	Existing responsibility	Connection examined here
Authorization languages [5]	Decide principal/action/resource requests	Bind a decision to candidate bytes, history, validators, and full context
Runtime monitors [4]	Mediate enforceable events	Recheck one lifecycle identity at admission, dispatch, and library exposure
Provenance models [15]	Record derivations	Make transitive origins operational in withdrawal and recovery
Transactional publication	Publish changes atomically	Couple approval freshness with receipts and byte-bound execution
AgentSpec, Progent [6], [7]	Runtime rules; privilege-policy evolution	Artifact-version/origin eligibility and publication/reuse contract
AutoSpec [8]	Learn safety rules	Protect policy/verifier state while mediating harness transitions
Self-evolving agents [1], [2], [3]	Propose and evaluate changes	Independent operator authority around the original adaptive loop

6 Conclusion

Approval in a self-evolving harness should be a versioned relation to a trusted context, not a property of bytes that passed a test once. SSEVOH-DSL binds each transition to state, policy, evidence, and origins, and re-enforces that binding at commit, execution, and library reuse. Bounded exploration, TLC cross-checks, and controlled Harness^2 fixtures show that the contract is implemented consistently and blocks violations that original admission, testing, and static validation miss, at sub-millisecond admission cost. The same evaluation shows what the contract does not yet deliver: the governed ledger agent learns from its library, and sandboxed evidence admits its correct repairs, but strict task success is still unestablished. Remaining priorities are complete frozen HumanEval+ results, professional tasks, durable recovery, verified isolation for evolving executable tools, and a mechanized refinement argument.

Data and Artifact Availability

The repository is the accompanying artifact (an immutable public release is required at submission). Readiness records list checked source files and hashes; `make verify` and integrations/harness2/scripts/verify.py rerun assurance without submitting experiments; docs/TSE/figures/make_figures.py regenerates Figs. 2–3. Table 9 maps evidence to locations.

References

[1] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," *arXiv preprint arXiv:2408.08435*, 2024. [Online]. Available: https://arxiv.org/abs/2408.08435

TABLE 9
Evidence-to-artifact map (repository-relative).

Evidence	Location
Language, core	docs/language/grammar.ebnf; src/ssevoh/
Assurance	formal/assurance/results/readiness.json; formal/model/
Integration	integrations/harness2/adapter/ssevoh_harness2/; upstream-runtime.json
DSL cohort	artifacts/benchmark-results/full-campaign/records.jsonl
Baselines, scaling	artifacts/benchmark-results/full-campaign/auxiliary.jsonl
Harness^2 studies	artifacts/benchmark-results/harness2-full/ (controlled.jsonl, model-streams.jsonl)
Ledger pilot v2	artifacts/benchmark-results/harness2-pilot-v2/; ToolRepair: experiments/toolrepair/
Qualification	artifacts/benchmark-results/qualification/; artifacts/manifests/hpc-final-audit.json
HumanEval+	experiments/humaneval/ (dataset-manifest.json, protocol.json)

[2] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint arXiv:2505.22954*, 2025. [Online]. Available: https://arxiv.org/abs/2505.22954

[3] R. Xu, Y. Chen, Z. CuiZhu, B. D. Mishra, Y. Ming, H. Yu, R. Han, C.-Y. Lee, and T. Pfister, "Harness^2: Recursive agent harnessing for an open world," *arXiv preprint arXiv:2610.14224*, 2026. [Online]. Available: https://arxiv.org/abs/2610.14224

[4] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, Feb. 2000. [Online]. Available: https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf

[5] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells, "Cedar: A new language for expressive, fast, safe, and analyzable authorization (extended version)," *arXiv preprint arXiv:2403.04651*, 2024. [Online]. Available: https://arxiv.org/abs/2403.04651

[6] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026, pp. 2938–2950. [Online]. Available: https://arxiv.org/abs/2503.18666

[7] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," *arXiv preprint arXiv:2504.11703*, 2026, version 3, revised May 2026; first submitted 2025. [Online]. Available: https://arxiv.org/abs/2504.11703v3

[8] P. Ma, Z. Wang, Z. Ji, Y. Zhou, Z. Xue, Z. Li, S. Wang, and X. Zhang, "AutoSpec: Safety rule evolution for LLM agents via inductive logic programming," *arXiv preprint arXiv:2606.24245*, 2026. [Online]. Available: https://arxiv.org/abs/2606.24245

[9] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[10] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.01210

[11] ai4se4ai-lab, "use-hpc," Software repository, 2026, pipeline base revision f9dc7d66ad8df3c202f6b72b02630e14fd148756. [Online]. Available: https://github.com/ai4se4ai-lab/use-hpc

[12] C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, J. Kastner, A. Mamat, M. McCutchen, N. Rungta, B. Shah, E. Torlak, and A. Wells, "How we built cedar: A verification-guided approach," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024. [Online]. Available: https://arxiv.org/abs/2407.01688

[13] G. C. Necula, "Proof-carrying code," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1997, pp. 106–119.

[14] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. [Online]. Available: https://lamport.azurewebsites.net/tla/book.html

[15] L. Moreau and P. Missier, "PROV-DM: The PROV data model," W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/